

# Lecture #4

---

Instructor: Yunseok Choi  
(ys.choi@skku.edu)

# Logistic Regression

---

# 로지스틱 회귀 분류기 Logistic Regression

- 로지스틱 회귀 분류기는 로지스틱 함수(시그모이드 함수)를 사용하는 분류기
- 로지스틱 함수 (Logistic Function)
  - = 시그모이드 함수 (Sigmoid Function)
  - 입력된 값에 0과 1사이 값을 할당하는 함수

$$y(z) = \frac{1}{1 + \exp(-z)}$$

# 로지스틱 함수 Logistic Function

## ■ Python에서 로지스틱 함수 구현

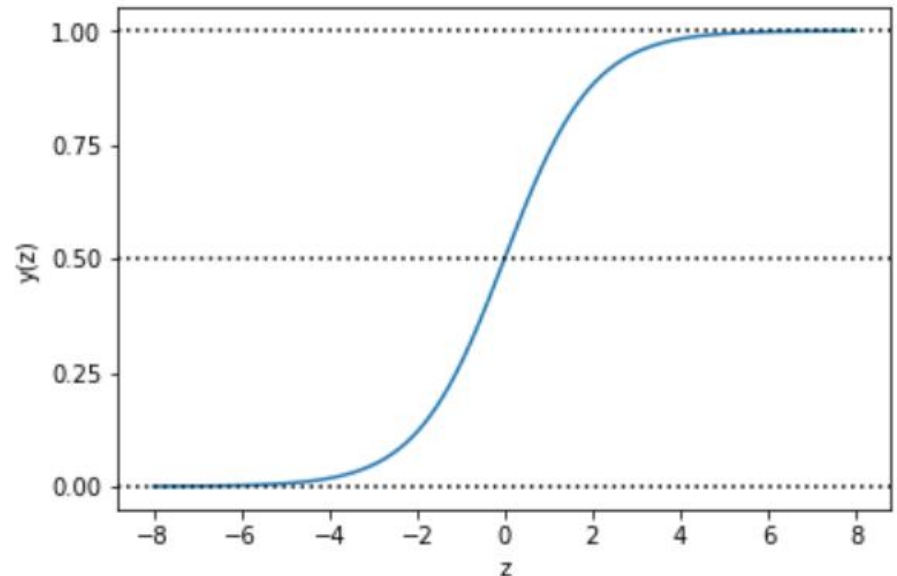
- -8 ~ 8 값을 넣었을 때, 0 ~ 1 값으로 변환된다.
- 입력이 0일 때 함수 값은 딱 중간 값, 0.5가 된다.

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(input):
    return 1.0 / (1 + np.exp(-input))

z = np.linspace(-8, 8, 1000)
y = sigmoid(z)

plt.plot(z, y)
plt.axhline(y=0, ls='dotted', color='k')
plt.axhline(y=0.5, ls='dotted', color='k')
plt.axhline(y=1, ls='dotted', color='k')
plt.yticks([0.0, 0.25, 0.5, 0.75, 1.0])
plt.xlabel('z')
plt.ylabel('y(z)')
plt.show()
```



# 피쳐 변환

- 로지스틱 회귀에서 함수의 입력인  $z$ 는 피쳐  $\mathbf{x}$ 의 가중치 합  
(입력 피쳐가 선형 변환된 값)

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n = \mathbf{w}^T \mathbf{x}$$

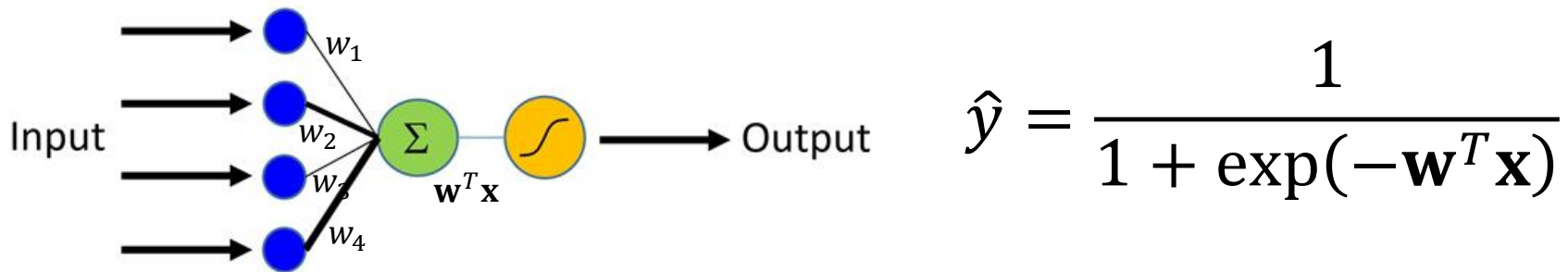
- 주어진 데이터를 선형 변환만으로 완벽하게 목적하는 공간으로 매핑하는 것은 어렵기 때문에 보통 절편(=바이어스)를 식에 추가해 사용

$$z = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n = \mathbf{w}^T \mathbf{x}$$

(내적으로 표현하기 위해  $\mathbf{x}$ 에 상수 1을 가지는 피쳐를 추가해 사용)

# 로지스틱 회귀 분류기

- 로지스틱 함수를 통한 피쳐 벡터  $\mathbf{x}$ 의 클래스 판별



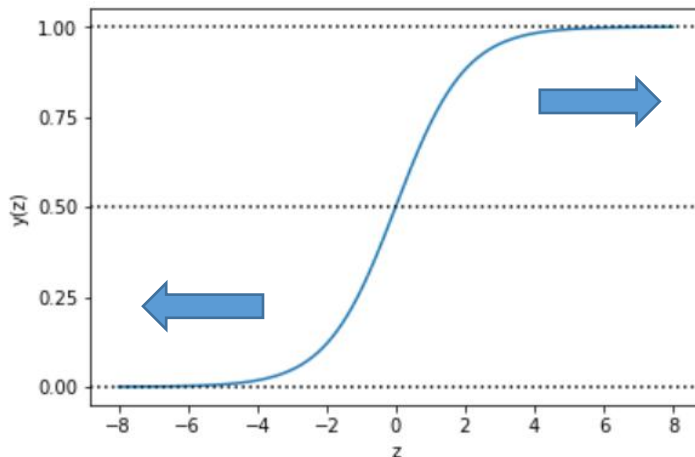
- 로지스틱 함수가 0 ~ 1 값을 가지기 때문에  $\hat{y}$ 를  $P(y = 1 | \mathbf{x})$  확률로 생각할 수도 있다.
  - 예) 확률이 0.5 보다 크면 1이라 판단하고, 0.5 보다 작으면 0이라 판단

# 피쳐 변환을 위한 가중치 $w$

- 로지스틱 회귀 분류 방법에서 학습은

피쳐를 잘 변환해 줄 수 있는 가중치  $w$ 를 찾아 내는 것

- (잘 변환 한다는 의미) 좋은 가중치일 수록 입력 피쳐값을
  - 음성 클래스에 대해서는 아주 작은 값( $\hat{y}$ 가 0에 가깝도록)으로,
  - 양성 클래스에 대해서는 아주 큰 값( $\hat{y}$ 가 1에 가깝도록)으로 변환



즉, 예측치(클래스 확률)와  
실측치(클래스 레이블)이  
최대한 같게 만들 수록  
좋은 가중치

# LR – Cost Function

---

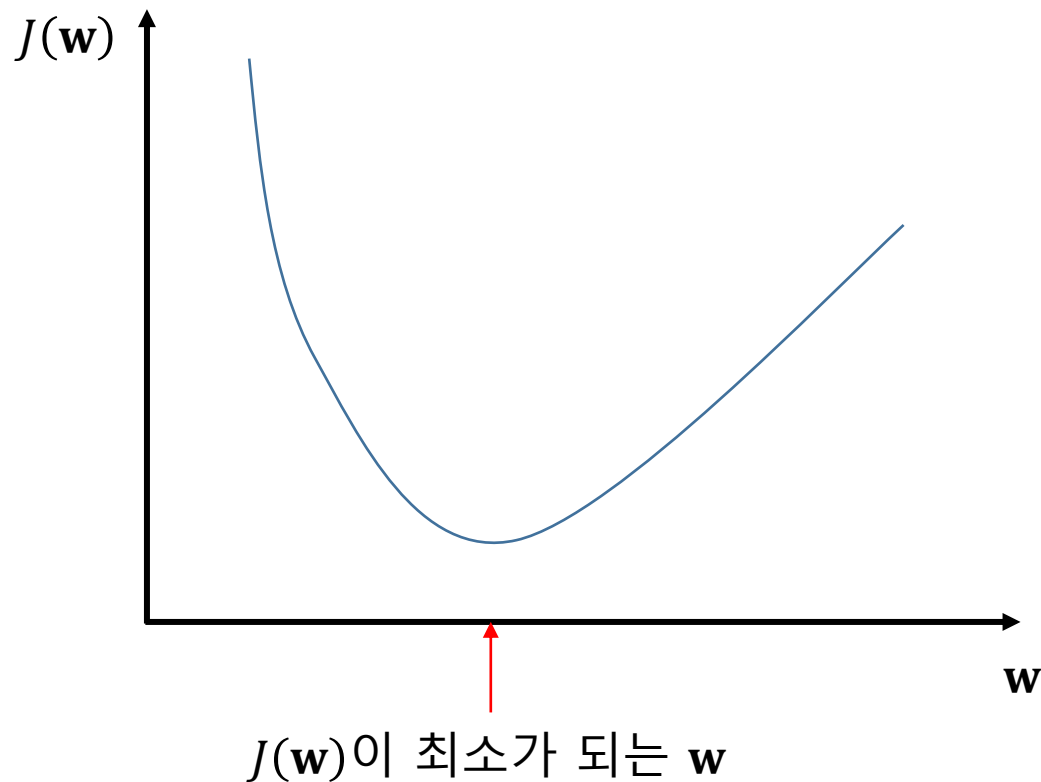


# Cost Function

- Cost Function은 최적의 가중치를 찾기 위해 학습 데이터에 대해서 예측치  $\hat{y}$ 와 실제값  $y$ 이 가지는 차이를 측정하는 함수
  - 학습 모델을 따르면 얼마의 비용(Cost)를 감당해야 하는지 나타낸다.
- Mean Squared Error (MSE) : 대표적인 Cost Function
  - 예측치  $\hat{y}$ 와 실제값  $y$ 의 차이를 제곱한 값의 평균

$$J(\mathbf{w}) = MSE(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2$$

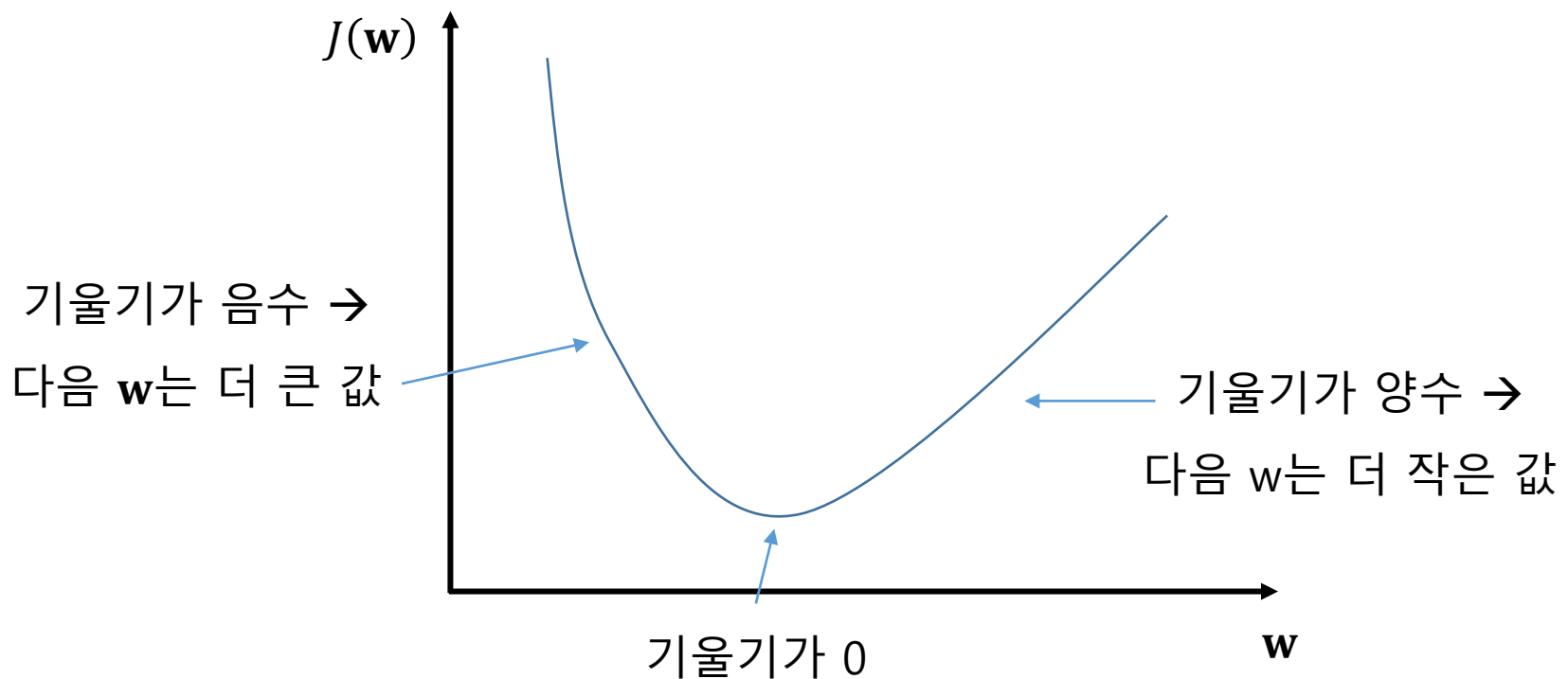
# Cost를 최소화 하는 $w$ 찾기



피쳐 공간이 복잡할 수록 모든  $w$  조합에 대한  $J(w)$ 값들을 다 짚어보는 것은 힘들다.

# 경사 하강법 Gradient Descent

- $w$ 를 바꿔가면서  $J(w)$ 를 찾는데, 다음  $w$ 를 결정할 때 기울기를 사용
  - 기울기가 양수면 다음번  $w$ 는 지금보다 작은 값을 사용
  - 기울기가 음수면 다음번  $w$ 는 지금보다 큰 값을 사용
  - 기울기가 0이면 최적값을 찾은 것



# Cost Function

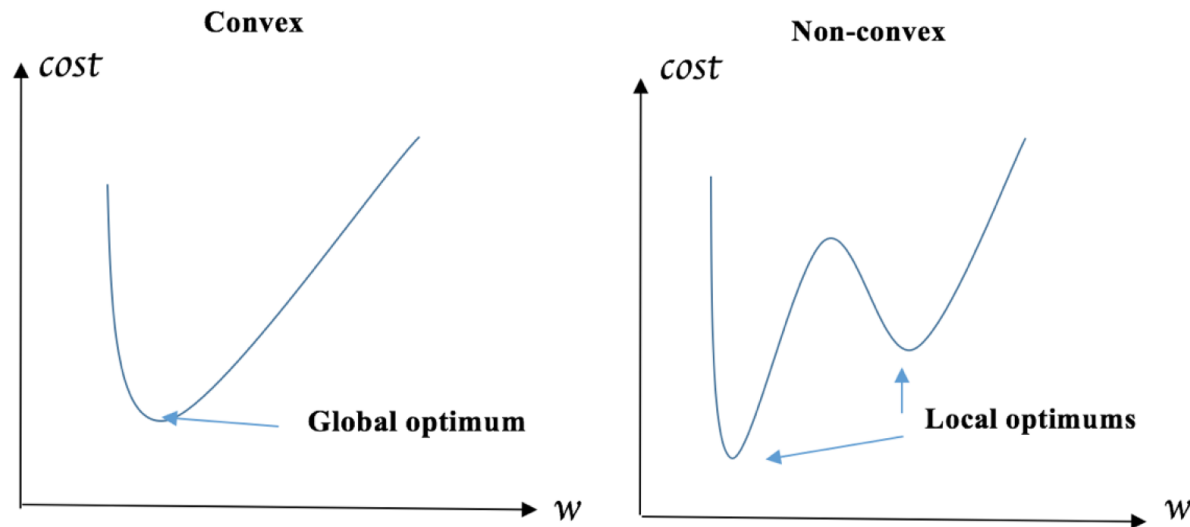
- 기울기를 구하기 위해서 미분을 사용하기 때문에 Cost 함수에  $\frac{1}{2}$ 를 추가해 깔끔한 계산이 되도록 한다.

$$J(\mathbf{w}) = MSE(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2$$

# MSE를 Cost Function으로 쓸 때 문제

$$J(\mathbf{w}) = MSE(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2$$

MSE는 볼록(Convex) 함수가 아니라 비볼록(Non-Convex) 함수라서 전역적 최적치(Global optimum)이 유일하게 존재하는 것이 아니라 국지적 최적치(Local optimum)들이 여러 개 존재할 수 있다.



# Cost Function : Binary Cross Entropy

- 이러한 문제를 극복하기 위해 엔트로피를 사용하는 Binary Cross Entropy를 Cost Function으로 사용한다.
  - 엔트로피가 Log를 사용하기 때문에 MSE보다 Non-Convex 문제에 더 잘 대응

$$J(\mathbf{w}) = BCE(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n -[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

(엔트로피와 비슷하게)

클래스 양쪽에 확률이 반반 될 수록

Cost 값이 커지고, 한쪽 클래스에 쏠릴 수록 값이 작아진다.

# LR – Gradient Descent

---

# 경사 하강법 Gradient Descent

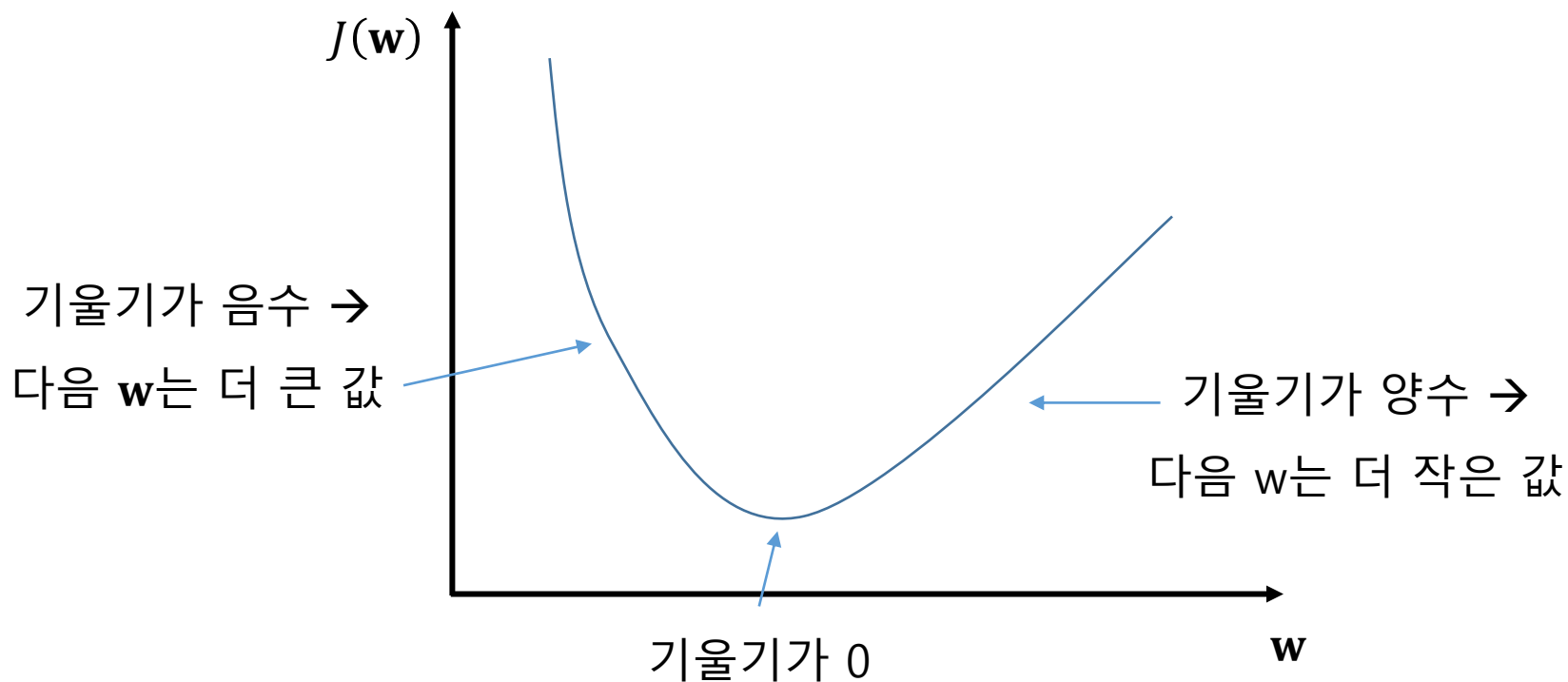
- Binary Cross Entropy 함수를 Cost Function으로 사용했을 때, 경사 하강법을 적용하여 최적치를 찾는 과정을 좀더 자세히 알아보자.

$$J(\mathbf{w}) = BCE(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n -[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$



# 경사 하강법 Gradient Descent

- $w$ 를 바꿔가면서  $J(w)$ 를 찾는데, 다음  $w$ 를 결정할 때 기울기를 사용
  - 기울기가 양수면 다음번  $w$ 는 지금보다 작은 값을 사용
  - 기울기가 음수면 다음번  $w$ 는 지금보다 큰 값을 사용
  - 기울기가 0이면 최적값을 찾은 것



# 경사 하강법 Gradient Descent

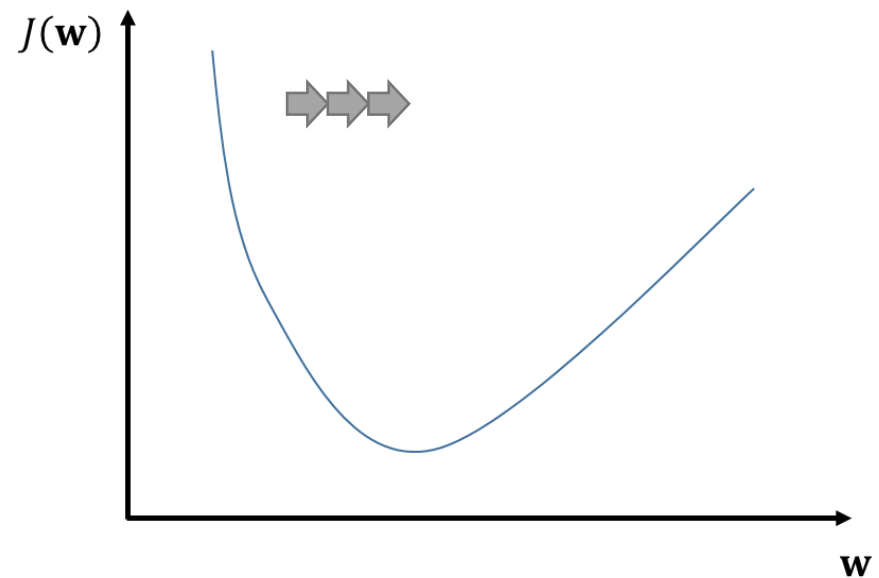
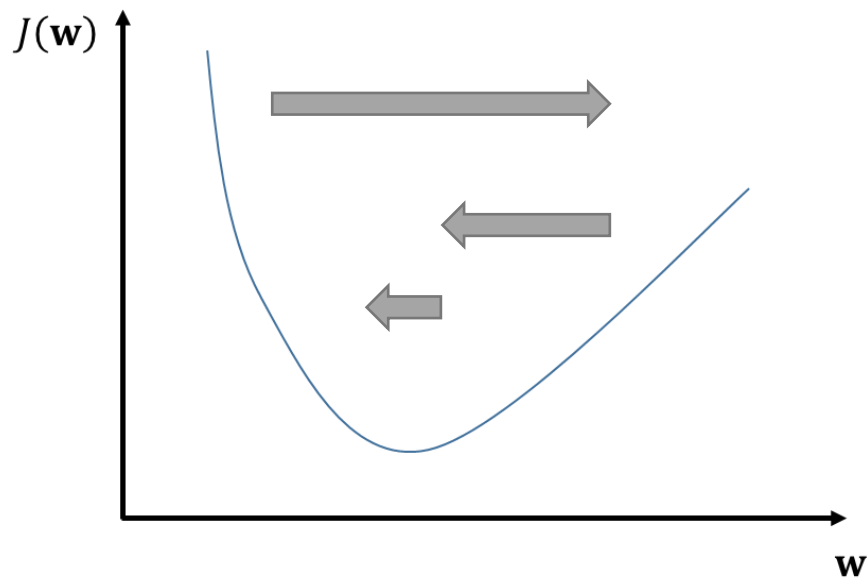
- 경사하강법은 최적화 대상 변수 값을  
목적 함수(Objective Function)의 가장 작은 값을 향해  
반복적으로 움직이는 방법
  - 로지스틱 회귀 문제에서 목적함수는 비용함수와 동일

$$\mathbf{w}' := \mathbf{w} - \eta \Delta \mathbf{w}$$

- $\Delta \mathbf{w}$  : 기울기 (= Gradient),  $J(\mathbf{w})$ 를 미분한 결과
- $\eta$  : 학습률 (Learning rate) or Step Size

# 학습률 Learning Rate

학습률은 그래디언트 값을 얼마나 반영해서  
가중치를 업데이트할지 조절하는 변수



# 기울기 Gradient

$\mathbf{w} = (w_1, w_2, \dots, w_k)$  이므로  
각 피쳐  $w_k$  별로 값이 업데이트되기 위해서는  
피쳐 별로 기울기를 구해야 한다.

일단  $w_k$ 에 대한 기울기  $\Delta w_k$ 를 구해보자.

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n -[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

$$\Delta w_k = \frac{d}{dw_k} J(w_k) = \frac{d}{dw_k} \frac{1}{n} \sum_{i=1}^n -[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

# 기울기 Gradient

$$\begin{aligned}\Delta w_k &= \frac{d}{dw_k} J(w_k) = \frac{d}{dw_k} \frac{1}{n} \sum_{i=1}^n -[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] \\&= -\frac{1}{n} \sum_{i=1}^n \left[ \frac{d}{dw_k} y^{(i)} \log(\hat{y}^{(i)}) + \frac{d}{dw_k} (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] \\&= -\frac{1}{n} \sum_{i=1}^n \left[ y^{(i)} \frac{d}{dw_k} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \frac{d}{dw_k} \log(1 - \hat{y}^{(i)}) \right] \\&= -\frac{1}{n} \sum_{i=1}^n \left[ \frac{y^{(i)}}{\hat{y}^{(i)}} - \frac{1-y^{(i)}}{1-\hat{y}^{(i)}} \right] \frac{d}{dw_k} \hat{y}^{(i)}\end{aligned}$$

# 기울기 Gradient

$$\frac{d}{dw_k} \hat{y} = \frac{d}{dw_k} \frac{1}{1+\exp(-z)}$$

$$= \frac{d}{dw_k} z \cdot \frac{d}{dz} \frac{1}{1+\exp(-z)} \quad \left( \left( \frac{1}{g(x)} \right)' = -\frac{g'(x)}{(g(x))^2} \right)$$

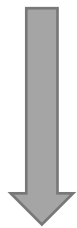
$$= -x_k \frac{d}{dz} (1 + \exp(-z)) \frac{1}{(1+\exp(-z))^2}$$

$$= x_k \frac{\exp(-z)}{(1+\exp(-z))^2}$$

$$= x_k \left[ \frac{1}{1+\exp(-z)} \right] \left[ 1 - \frac{1}{1+\exp(-z)} \right] = x_k \hat{y}(1 - \hat{y})$$

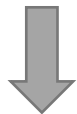
# 기울기 Gradient

$$\Delta w_k = -\frac{1}{n} \sum_{i=1}^n \left[ \frac{y^{(i)}}{\hat{y}^{(i)}} - \frac{1-y^{(i)}}{1-\hat{y}^{(i)}} \right] \frac{d}{dw_k} \hat{y}^{(i)}$$



$$\frac{d}{dw_k} \hat{y} = x_k \hat{y}(1 - \hat{y})$$

$$\Delta w_k = \frac{1}{n} \sum_{i=1}^n (-y^{(i)} + \hat{y}^{(i)}) x_k^{(i)}$$



모든 피처에 대한 일반화 된 식으로 표현

$$\Delta \mathbf{w} = \frac{1}{n} \sum_{i=1}^n (-y^{(i)} + \hat{y}^{(i)}) \mathbf{x}^{(i)}$$

# 경사 하강법 Gradient Descent

업데이트를 여러 번 반복하여  $\Delta \mathbf{w}$ 에 큰 변화가 없는 상태에 도달하면 정지

$$\mathbf{w}' := \mathbf{w} - \eta \frac{1}{n} \sum_{i=1}^n (-y^{(i)} + \hat{y}^{(i)}) \mathbf{x}^{(i)}$$

$\eta$  : 학습률 (Learning rate) or Step Size



# 경사 하강법의 단점

- 로지스틱 회귀 모델에서  
그레디언트 하강 기법은 매번 반복 시행 때마다 가중치를 업데이트 하기 위해 학습 데이터 전체를 사용!
- 때문에 학습 데이터가 커지면 커질 수록 학습에 소요되는 시간과 계산 비용이 엄청나게 커진다.

# 스토캐스틱 그레디언트 하강 (SGD)

- 스토캐스틱 그레디언트 하강(Stochastic Gradient Descent)은  
가중치 업데이트 과정에서 학습 데이터 전체를 사용하는 것  
대신 샘플 데이터 한 개만 사용한다.
- 즉, 학습 샘플 데이터 한 개로 계산한 에러 값을  
바탕으로 가중치를 업데이트 하고 다음으로 넘어간다.
  - 한 개 씩 해서 전체 학습 데이터가 한번 씩 반영됐다면  
학습 반복(epoch)이 한번 수행 됐다고 간주한다.
- 보통 로지스틱 회귀의 경우 epoch이 10번 이내에서 수렴  
(보통 기본 Gradient Descent에 비해 반복 횟수가 더 적다)

# 미니배치 그레디언트 하강

- 전통적인 SGD 방법은 **학습 데이터의 샘플 한 개씩** 돌아가며 에러를 계산하고 가중치를 업데이트
- **미니배치 그레디언트 하강은 샘플을 여러 개 사용**하는 방법
  - 메모리에 넣을 수 있는 정도는 여러 개 쓰자
  - 즉, 배치 크기가 32이면 32개 샘플을 가지고 그레디언트 평균을 구하고 이 값을 가지고 가중치를 한번 업데이트
- SGD 방법의 일반화 버전으로 보기도 한다.  
(배치 크기를 1로 잡으면 전통적인 SGD와 동일 하기에...)

# CTR 예측: SGD 적용

```
from sklearn.linear_model import SGDClassifier

parameters = {'eta0': [0.1, 0.01, 0.001]}

sgd_lr = SGDClassifier(loss='log', fit_intercept=True)

grid_search = GridSearchCV(sgd_lr, parameters,
                           n_jobs=-1, cv=3, scoring='roc_auc')

grid_search.fit(onehot_train_X, train_y)

sgd_lr_best = grid_search.best_estimator_
pos_prob = sgd_lr_best.predict_proba(onehot_test_X)[:, 1]
print('The ROC AUC on testing set is :{0:.3f}'
      .format(roc_auc_score(test_y, pos_prob)))
```

The ROC AUC on testing set is :0.694

# LR – Regularization

---

# 정규화 Regularization

- 지금까지 봤던 목적 함수(=비용 함수)는 오버피팅의 위험 내포
  - 즉, 학습 데이터에 대해서 Cost 값을 아주 작게 만드는  $\mathbf{w}$ 를 찾았는데 이  $\mathbf{w}$ 가 실제 일반 데이터에는 잘 맞지 않을 수 있다.
- 어떤 경우에 그럴까?
  - $\mathbf{z}$ 는  $\mathbf{w} \cdot \mathbf{x}$ 로 계산되는데 이때  $\mathbf{x}$  값보다  $\mathbf{w}$  값들이 엄청나게 커질 수 있다.  
( $\mathbf{x}$ 는 입력 데이터이기 때문에 어느정도 가질 수 있는 값의 범위가 있다.)
  - $\mathbf{w}$ 가 엄청 커지면 입력 데이터 값에 변화에 둔감해 질 수 있다.
  - 즉,  $\mathbf{x}$  데이터 무시하고 Cost(학습 데이터 에러)만 줄일 수 있으면 그 방향으로 학습할 수도 있다.

어떻게 해결할 수 있을까?

# 정규화 Regularization

- 기본적으로는 목적함수로 정의된 Cost를 줄이는 방향으로 학습을 하되, 되도록이면 작은  $w$  값을 사용할 수 있도록!
  - Cost에  $w$  값 자체 크기를 추가해 준다. (추가 에러)  
 $w$  값의 크기가 커지면 Cost가 증가하는 Penalty를 받는다.
- ➔ 이렇게 오버피팅을 방지하기 위해  
여분의 에러를 의도적으로 추가하는 것을 "정규화"

# 정규화 Regularization

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n -[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] + \alpha \|\mathbf{w}\|^q$$

- $\alpha$ 는 정규화를 얼마나 목적함수에 반영할지 조절하는 파라미터
- 정규화는 벡터의 Norm(크기)를 보통 사용

- L1 정규화 : L1 Norm을 사용

$$\|\mathbf{w}\|^1 = \sum_{i=1}^k |w_i|$$

- L2 정규화 : L2 Norm을 사용

$$\|\mathbf{w}\|^2 = \sum_{i=1}^k w_i^2$$



# 정규화 in Python

```
from sklearn.linear_model import SGDClassifier
parameters = {'eta0': [0.1, 0.01, 0.001]}

sgd_lr = SGDClassifier(loss='log', fit_intercept=True, penalty='l1')

grid_search = GridSearchCV(sgd_lr, parameters,
                           n_jobs=-1, cv=3, scoring='roc_auc')

grid_search.fit(onehot_train_X, train_y)

sgd_lr_best = grid_search.best_estimator_
pos_prob = sgd_lr_best.predict_proba(onehot_test_X)[:, 1]
print('The ROC AUC on testing set is :{0:.3f}'
      .format(roc_auc_score(test_y, pos_prob)))
```

(그 외) alpha parameter로

정규화 factor 조절 가능

The ROC AUC on testing set is :0.692

penalty 파라미터로 옵션을 줄 수 있으며 아래 세 가지 중 하나를 선택 가능

'l1', 'l2', 'elasticnet' (L1과 L2를 혼합해 사용)

# LR – Multi-Classes

---

# 다중클래스 분류 처리

- 지금까지 로지스틱 회귀 알고리즘을 이진 분류에 적용해 봤지만, 어렵지 않게 다중 클래스 분류 문제로 확장시킬 수 있다.
- 3개 이상의 클래스에 대한 로지스틱 회귀를  
다중 로지스틱 회귀 (Multinomial Logistic Regression)  
이라 한다. (= 소프트맥스 회귀 Softmax Regression)

# 로지스틱 함수 vs. 소프트맥스 함수

- 이진 클래스 문제에서는 로지스틱 함수를 사용

$$\hat{y} = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x})}$$

- 다중 클래스 문제는 소프트맥스 함수를 사용

- $K$ 개 클래스가 있을 때 피쳐 벡터  $\mathbf{x}$ 에 대한 클래스  $k$ 에 대한 예측값 (클래스  $k$ 일 확률)

$$\hat{y}_k = \frac{\exp(\mathbf{w}_k^T \mathbf{x})}{\sum_j^K \exp(\mathbf{w}_j^T \mathbf{x})}$$

가중치  $\mathbf{w}$ 도 각 클래스 별로 존재한다.

$$\mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_3, \dots, \mathbf{w}_K$$

# 목적 함수 (비용 함수)

## ■ 이진 클래스

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n -[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

$y^{(i)}$ 이 1인 경우만 살도록  
(= 정답이 1인 경우)

$\log(\hat{y}^{(i)})$  값을 반영  
높은 확률로 1이 맞다고  
예측했을 수록  
높은 값을 반환

$y^{(i)}$ 이 0인 경우만 살도록  
(= 정답이 0인 경우)

$\log(1 - \hat{y}^{(i)})$  값을 반영  
낮은 확률로 1이 맞다고  
예측했을 수록  
높은 값을 반환

# 목적 함수 (비용 함수)

- 다중 클래스

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n - \left[ \sum_{j=1}^K 1\{y^{(i)} = j\} \log(\hat{y}_j^{(i)}) \right]$$

(  $1\{y^{(i)} = j\}$ 은  $y^{(i)} = j$ 일 때 1을 반환, 그 외 경우는 0을 반환하는 함수 )

각 학습 데이터 샘플의

해당 클래스에 대한 예측값(확률)을 비용에 반영

# 그레디언트

## ■ 이진 클래스

$$\Delta \mathbf{w} = \frac{1}{n} \sum_{i=1}^n (-y^{(i)} + \hat{y}^{(i)}) \mathbf{x}^{(i)}$$

## ■ 다중 클래스

$$\Delta \mathbf{w}_k = \frac{1}{n} \sum_{i=1}^n (-1\{y^{(i)} = k\} + \hat{y}_k^{(i)}) \mathbf{x}^{(i)}$$

식을 유도하는 것도

이진 클래스와 비슷하게 하면 된다.

- $y^{(i)}$ 가  $k$  일 때  $k$ 에 대한 확률이 높거나,  
 $y^{(i)}$ 가  $k$ 가 아닐 때  $k$ 에 대한 확률이 낮으면 그레디언트가 0에 가까워짐  
(잘 맞췄으므로 업데이트 필요 없다는 의미)

# 학습과 최종 예측

- 각 가중치 별로 계산되는 그래디언트에 따라서  
가중치 벡터들을 반복적으로 업데이트 하고,  
전체 비용 함수값에 큰 변화가 없게 되면 (수렴) 정지
- 학습된 가중치 벡터들을 가지고 아래 식을 이용해 최종 예측
  - 예측값이 가장 큰 클래스를 선택

$$y' = \operatorname{argmax}_k \hat{y}_k$$



# 뉴스 그룹 분류에 적용해 보기

- Newsgroup20 데이터를 주제 별로 (20개 클래스) 분류해 보자.
  - 전처리기는 앞에서 구현한 것을 사용하자.

```
from nltk.corpus import names
from nltk.stem import WordNetLemmatizer

all_names = set(names.words())
lemmatizer = WordNetLemmatizer()

def clean_text(docs):
    cleaned_docs = []
    for doc in docs:
        lemmatized_list = [ lemmatizer.lemmatize( word.lower() )
                             for word in doc.split()
                             if word.isalpha() and word not in all_names ]
        cleaned_docs.append(' '.join(lemmatized_list))

    return cleaned_docs
```

# 뉴스 그룹 분류에 적용해 보기

## ■ 데이터 읽어오기

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer

data_train = fetch_20newsgroups(subset='train',
                                categories=None, random_state=42)

data_test = fetch_20newsgroups(subset='test', categories=None,
                                random_state=42)

cleaned_train = clean_text(data_train.data)
label_train = data_train.target
cleaned_test = clean_text(data_test.data)
label_test = data_test.target
tfidf_vectorizer = TfidfVectorizer(sublinear_tf=True,
                                   max_df=0.5, stop_words='english', max_features=40000)
term_docs_train = tfidf_vectorizer.fit_transform(cleaned_train)
term_docs_test = tfidf_vectorizer.transform(cleaned_test)
```

# 뉴스 그룹 분류에 적용해 보기

- 모델 학습 (하이퍼 파라미터 탐색)

```
from sklearn.model_selection import GridSearchCV

parameters = {'penalty': ['l2', None],
              'alpha': [1e-07, 1e-06, 1e-05, 1e-04],
              'eta0': [0.01, 0.1, 1, 10]}

sgd_lr = SGDClassifier(loss='log', learning_rate='constant',
                      eta0=0.01, fit_intercept=True, n_iter=10)

grid_search = GridSearchCV(sgd_lr, parameters,
                          n_jobs=-1, cv=3)

grid_search.fit(term_docs_train, label_train)

print(grid_search.best_params_)

{'alpha': 1e-07, 'eta0': 10, 'penalty': None}
```

# 뉴스 그룹 분류에 적용해 보기

- 테스트셋에 대한 최종 평가

```
sgd_lr_best = grid_search.best_estimator_  
  
accuracy = sgd_lr_best.score(term_docs_test, label_test)  
  
print('The accuracy on testing set is: {0:.1f}%'.format(accuracy*100))
```

The accuracy on testing set is: 79.5%

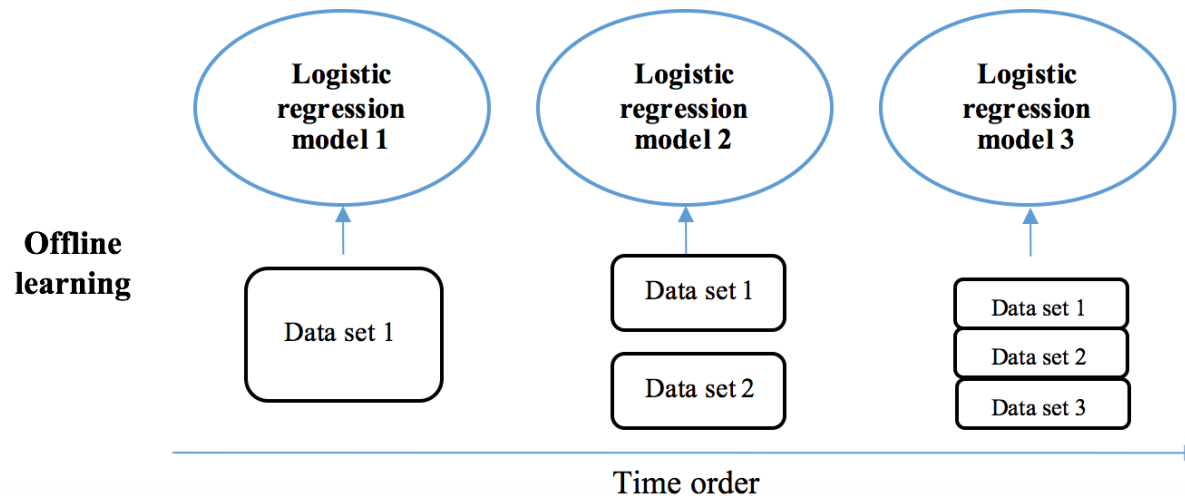
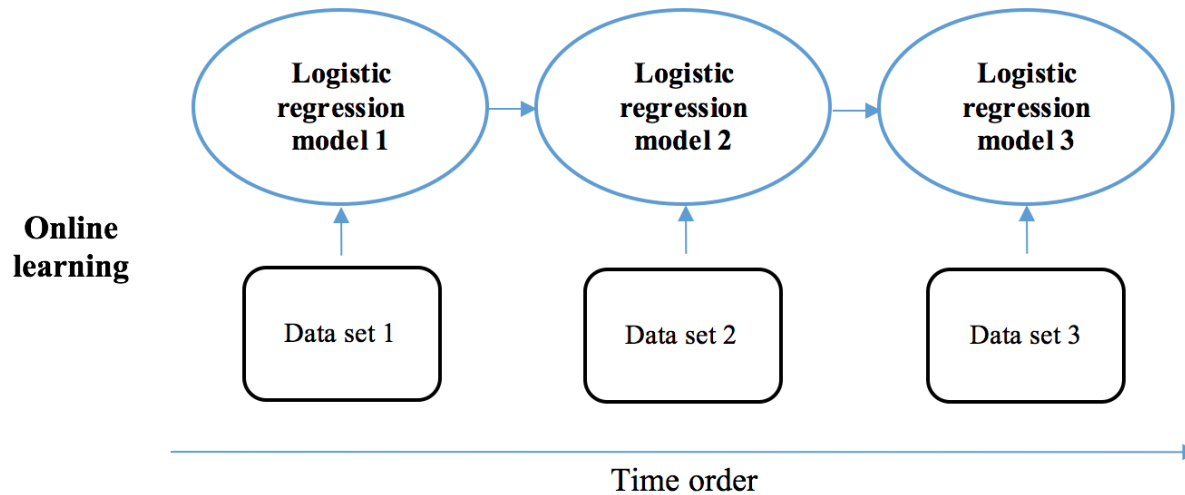
# LR – Online Learning

---

# 온라인 러닝 Online Learning

- 스토캐스틱 경사 하강법이 어차피 전체 데이터를 한번에 필요로 하는 것은 아니기 때문에 굳이 전체 데이터를 항상 메모리에 유지하고 있을 필요는 없다.
- **온라인 러닝**은 학습 데이터 전체를 처음부터 다 불러다 사용하는 것이 아니라, **일부분씩 순차적**으로 읽어와서 학습에 사용하는 기법
  - ➔ 대규모 데이터 전체를 메모리 상에 유지 할 필요 X
  - ➔ 데이터가 실시간으로 계속 들어오는 경우에도 활용 가능

# 온라인 러닝 Online Learning



# Lab 4-1: Logistic Regression

---



# Lab

---

- NH Data Science Course

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
  - 김규태
  - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.

# Regression Basic

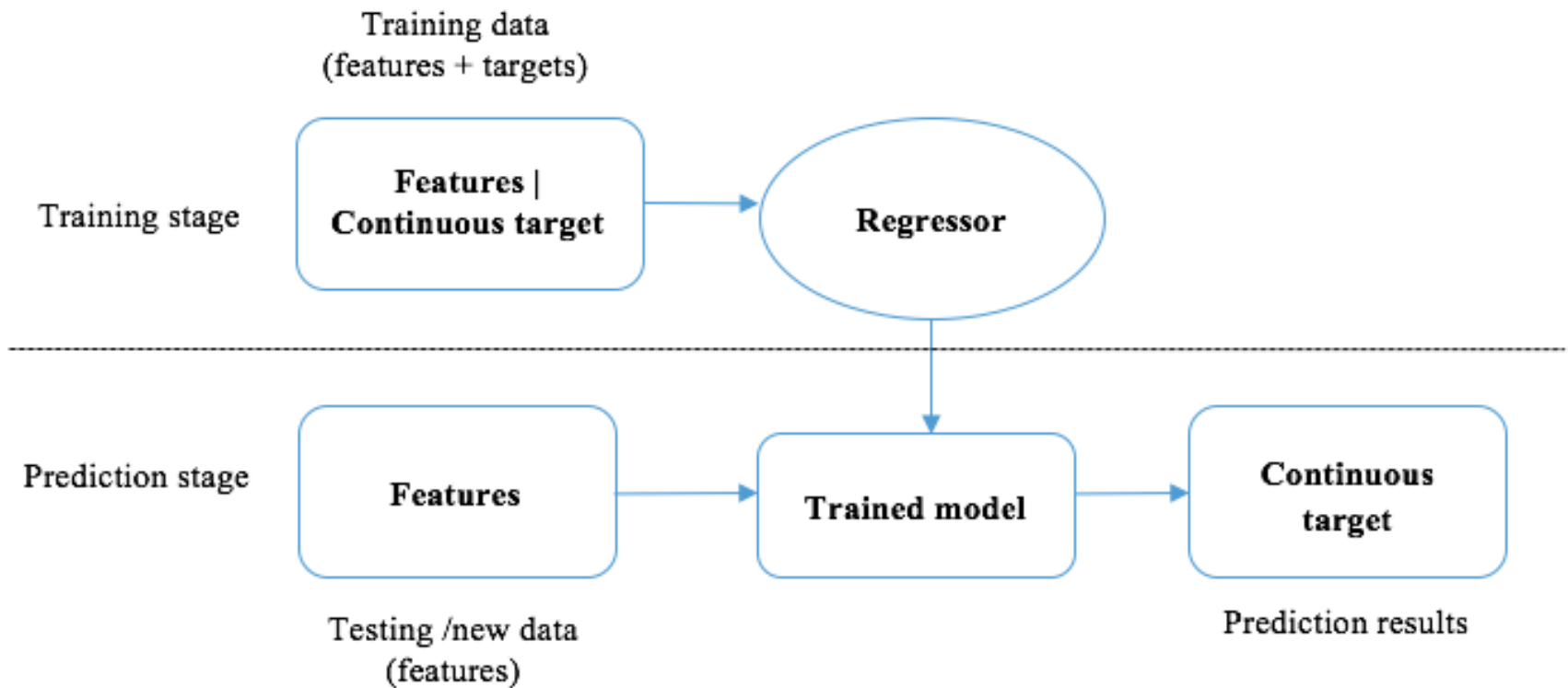
---

Instructor: Yunseok Choi  
([ys.choi@skku.edu](mailto:ys.choi@skku.edu))

# 회귀 Regression

- 분류와 같이 회귀도 지도학습의 대표적인 기법
- 분류와 달리 회귀는 관측값과 이에 대응되는 연속형 결과값 사이의 연관성을 찾는 것을 목표로 한다.
  - 결과값의 데이터 타입이 다른 것이 결정적 차이
  - 새로운 샘플에 대해서도 회귀는 그에 상응하는 연속형 예측값을 반환한다.

# 회귀 Regression



# 회귀 응용 분야

- 회귀는 정량적인 평가/응답을 예측하는 곳에 사용될 수 있다.
  - 주택 가격 예측 (지리적 위치, 평방 미터, 침실 개수, 욕실 개수 등을 고려)
  - 시스템의 프로세스와 메모리 정보를 바탕으로 한 전력 사용량 평가
  - 소매업 재고 예측
  - 주가 예측
  - ...

# 회귀 종류

---

- 선형 회귀 Linear Regression
- 의사결정 트리 회귀 Decision Tree Regression
- 서포트 벡터 회귀 Support Vector Regression

# Linear Regression

---

# 선형 회귀 Linear Regression

- 피처와 연속형 결과값 사이 관계를 설명하는 선형 방정식 (선형결합) 혹은 가중치 합의 함수를 찾는 알고리즘
- 입력 피처 벡터  $\mathbf{x} = (x_1, x_2, \dots, x_n)$  가 있고 이에 대응되는 결과값  $y$ 가 있을 때,  $y$ 를 가능한 잘 맞출 수 있는 선형 방정식을 찾는다.

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n = \mathbf{w}^T \mathbf{x}$$

$\hat{y}$ 가  $y$ 에 비슷할 수록 좋은 선형 회귀 모델이다.



# 선형 회귀 Linear Regression

- 하지만 실제 데이터가 선형 관계로 딱 표현되지 않을 수도 있기 때문에 절편(바이어스)를 추가해 준다.

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n = \mathbf{w}^T \mathbf{x}$$

- 어디서 많이 본 듯한 ...
  - 로지스틱 회귀에서 사용했던 식과 동일하다.
  - 다만, 로지스틱 회귀에서는 방정식의 결과값을 로지스틱 함수를 통해  
0~1 사이로 변환하는 과정이 더 존재했다.

# 목적 함수 (비용 함수)

- 기본적인 컨셉은 로지스틱 회귀에서 봤던 목적 함수와 동일

비용 = "결과값과 예측값의 차이"

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2$$

# 학습 : 그레디언트 하강 기법

- 그레디언트 하강 기법을 통해서  $J(\mathbf{w})$ 의 값이 최소가 되도록 하는  $\mathbf{w}$ 의 최적치를 구하게 한다.

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2$$



$$\Delta \mathbf{w} = \frac{1}{n} \sum_{i=1}^n (-y^{(i)} + \hat{y}^{(i)}) \mathbf{x}^{(i)}$$

유도 과정은 로지스틱 회귀 때와 비슷 (참고해서 직접 해보기)

# 학습 : 그레디언트 하강 기법

- 그레디언트 값을 이용해 매 단계마다 가중치 벡터를 업데이트 해 나간다.

$$\Delta \mathbf{w} = \frac{1}{n} \sum_{i=1}^n (-y^{(i)} + \hat{y}^{(i)}) \mathbf{x}^{(i)}$$



$$\mathbf{w} := \mathbf{w} + \eta \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)}) \mathbf{x}^{(i)}$$

# 예측

- 학습을 통해 가중치 벡터  $\mathbf{w}$ 를 찾게 되면,  
새로운 입력 벡터  $\mathbf{x}'$ 에 대한 결과 예측값  $y'$ 은  
다음과 같이 구할 수 있다.

$$y' = \mathbf{w}^T \mathbf{x}'$$

# 선형 회귀 in Python

- 사이킷런에 있는 당뇨 질환(diabetes) 데이터에 선형 회귀 모델을 적용해 보자.
  - 결과값 (타겟) : 1년 뒤 측정한 당뇨병의 진행률
  - 피처 (10종)
    - 나이, 성별, BMI, 혈압, 익명화된 피처 S1~S6  
(이 데이터셋의 특징 데이터는 모두 정규화된 값이다.)

```
from sklearn import datasets
diabetes = datasets.load_diabetes()
print(diabetes.data.shape)
```

(442, 10)

# 선형 회귀 in Python

- 학습 데이터 / 테스트 데이터 분리

```
num_test = 30 # the last 30 samples as testing set
X_train = diabetes.data[:-num_test, :]
y_train = diabetes.target[:-num_test]

X_test = diabetes.data[-num_test:, :]
y_test = diabetes.target[-num_test:]
```

- 선형 회귀 모델 학습

```
from sklearn.linear_model import SGDRegressor

regressor = SGDRegressor(loss='squared_loss', penalty='l2',
                          alpha=0.0001, learning_rate='constant', eta0=0.01, n_iter=1000)
```

# 선형 회귀 in Python

- 학습된 선형 회귀 모델로 예측

```
regressor.fit(X_train, y_train)
predictions = regressor.predict(X_test)

print(predictions)
```

```
[235.17654636 129.10331633 172.33882078 174.88242557 230.68418476
 159.03062487 107.92002457  93.56506816 149.8419903  195.07927133
 201.27904675 155.64309401 174.34095445 112.75792089 168.50839703
 138.3176281  263.4715931  107.24204583 121.78640184 126.43039563
 223.25754467  69.52910032 141.60186652 127.42924963  61.52892474
 195.25605476 113.35838388 132.47068983 211.30523225  55.23806536]
```



# Decision Tree Regression

---

# 의사결정 트리 회귀 Decision Tree Regression

## ■ (복습) 분류를 위한 의사결정 트리

- 각 노드에서 왼쪽 자식 노드와 오른쪽 자식 노드로 반복적으로 분할하고 확장시켜 트리 생성
- 각 파티션 단계에서 최적의 분할 지점이라고 판단되는 피처의 가장 주요한 조합을 탐욕적 탐색 방법으로 찾아감
  - 이 과정에서 어떤 조합이 분할을 잘 해내는지 측정하기 위한 기준으로 '지니 계수', '정보 이득' 기법 등이 있었음

의사결정 트리 회귀도 기본적으로는 분류 상황과 동일

타겟이 카테고리가 아니라 "연속형 변수"라는 점만 차이

# 의사결정 트리 회귀 Decision Tree Regression

## ■ 회귀를 위한 의사결정 트리

- 각 노드에서 왼쪽 자식 노드와 오른쪽 자식 노드로 반복적으로 분할하고 확장시켜 트리 생성
- 각 파티션 단계에서 최적의 분할 지점이라고 판단되는 피처의 가장 주요한 조합을 탐욕적 탐색 방법으로 찾아감
  - 지니계수나 정보이득 처럼 분할 포인트의 성능을 측정하기 위해 분할 결과로 얻어지는 **두 자식 노드의 가중 MSE**를 계산
  - 자식 노드들의 MSE는 타겟 값 전체의 분산과 같기 때문에, MSE가 작을 수록 분할이 더 잘 이뤄졌다는 의미가 된다.  
(작은 MSE → 특정 값으로 잘 모여있다.)

# 의사결정 트리 회귀 Decision Tree Regression

## ■ 회귀를 위한 의사결정 트리

- 각 노드에서 왼쪽 자식 노드와 오른쪽 자식 노드로 반복적으로 분할하고 확장시켜 트리 생성
- 각 파티션 단계에서 최적의 분할 지점이라고 판단되는 피처의 가장 주요한 조합을 탐욕적 탐색 방법으로 찾아감
  - 지니계수나 정보이득 처럼 분할 포인트의 성능을 측정하기 위해 분할 결과로 얻어지는 **두 자식 노드의 가중 MSE**를 계산
  - 분류에서는 마지막 리프 노드에 '클래스'를 할당했으나, 회귀에서는 마지막 **리프 노드에 평균값을 할당**한다.

# 의사결정 트리 회귀 예

- 피쳐 : Type, 침실 수
- 결과값 : 가격

| Type     | Number of bedrooms | Price (thousand) |
|----------|--------------------|------------------|
| Semi     | 3                  | 600              |
| Detached | 2                  | 700              |
| Detached | 3                  | 800              |
| Semi     | 2                  | 400              |
| Semi     | 4                  | 700              |

# 의사결정 트리 회귀 예

## ■ 가능한 모든 조합에 대한 MSE 계산

- $MSE(\text{type}, \text{semi})$
- $MSE(\text{type}, \text{detached})$
- $MSE(\text{bedroom}, 2)$
- $MSE(\text{bedroom}, 3)$
- $MSE(\text{bedroom}, 4)$

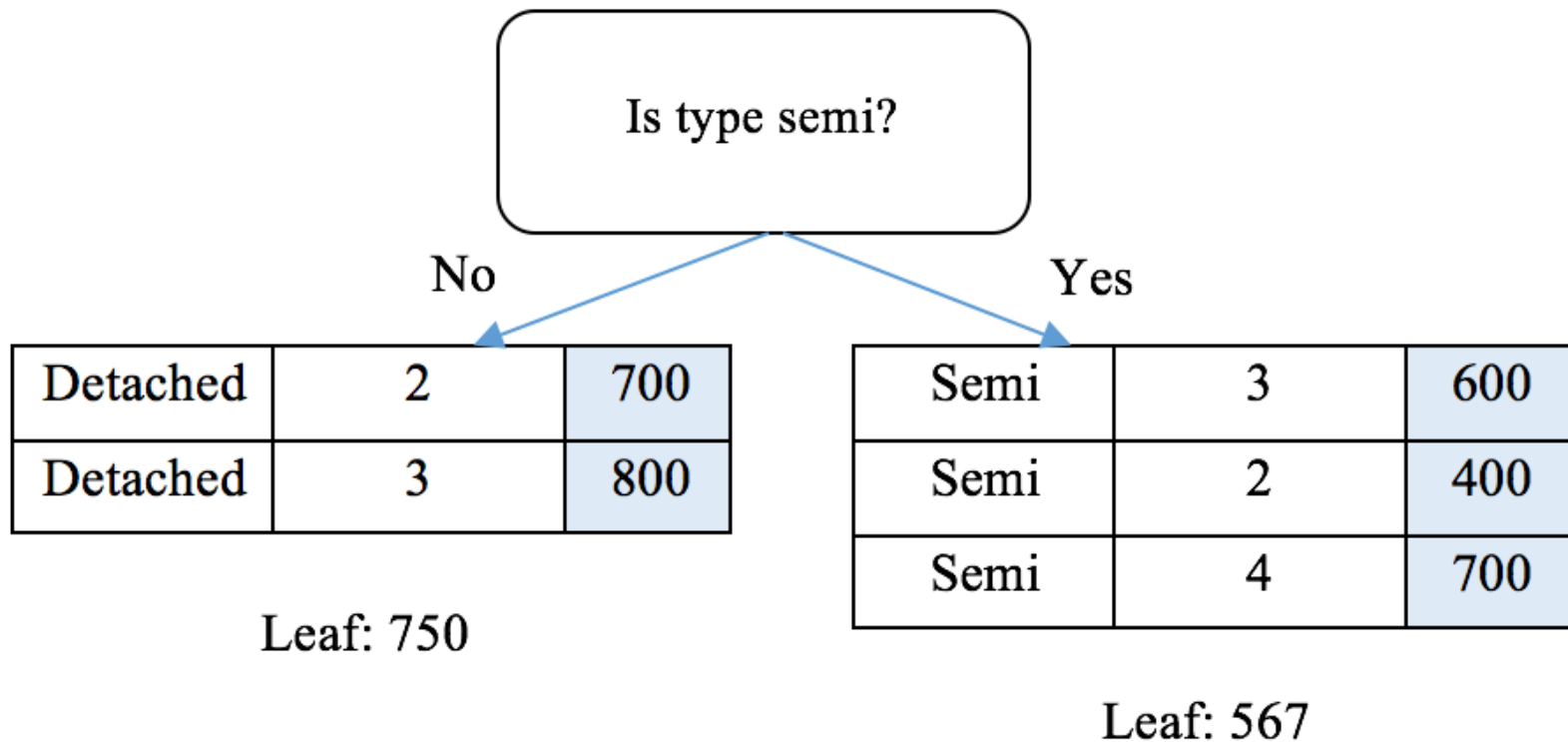
| Type     | Number of bedrooms | Price (thousand) |
|----------|--------------------|------------------|
| Semi     | 3                  | 600              |
| Detached | 2                  | 700              |
| Detached | 3                  | 800              |
| Semi     | 2                  | 400              |
| Semi     | 4                  | 700              |

# 의사결정 트리 회귀 예

- $\text{MSE}(\text{type}, \text{semi}) = \frac{3}{5} * \text{MSE}([600, 400, 700])$   
 $+ \frac{2}{5} * \text{MSE}([700, 800]) = 10333$
- $\text{MSE}(\text{type}, \text{detached}) \rightarrow \text{type이 두 종류밖에 없어서 semi와 동일}$
- $\text{MSE}(\text{bedroom}, 2) = \frac{2}{5} * \text{MSE}([700, 400])$   
 $+ \frac{3}{5} * \text{MSE}([600, 800, 700]) = 13000$
- $\text{MSE}(\text{bedroom}, 3) = \frac{4}{5} * \text{MSE}([600, 700, 800, 400])$   
 $+ \frac{1}{5} * \text{MSE}([700]) = 17500$
- $\text{MSE}(\text{bedroom}, 4) \rightarrow \text{모두다 4보다 같거나 작으므로 분할 X Pass}$

# 의사결정 트리 회귀 예

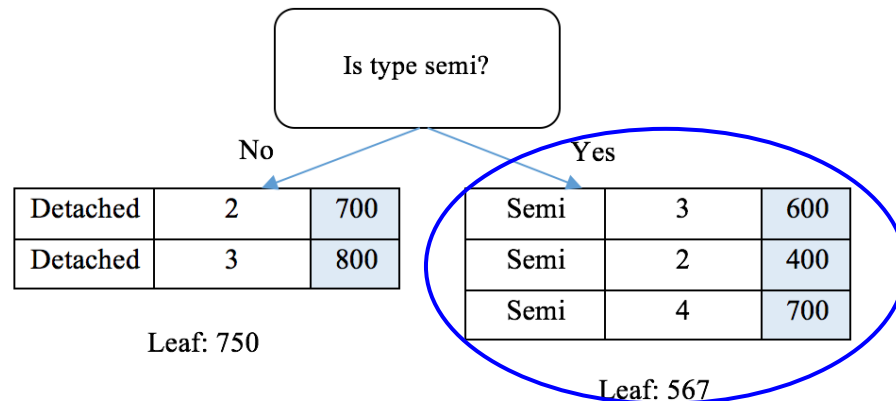
- MSE가 가장 작은 값을 가졌던 Type - Semi를 선택
  - 각 리프노드에는 **노드 데이터 평균**을 할당한다.





# 의사결정 트리 회귀 예

- 깊이가 1인 회귀 트리로 충분하다 생각하면 종료  
만약 트리 분할이 더 필요하다 생각하면 같은 방식을  
각 노드 별로 적용한다.
- 왼쪽 자식 노드의 샘플 수가 이미 2개이므로  
오른쪽 자식 노드에 대해서만 분할을 한번 더 해보자.



# 의사결정 트리 회귀 예

- $\text{MSE}(\text{bedroom}, 2) = 1/3 * \text{MSE}([400])$   
 $+ 2/3 * \text{MSE}([600, 700]) = 1667$
- $\text{MSE}(\text{bedroom}, 3) = 2/3 * \text{MSE}([600, 400])$   
 $+ 1/3 * \text{MSE}([700]) = 6667$
- $\text{MSE}(\text{bedroom}, 4) \rightarrow$  모두다 4보다 같거나 작으므로 분할 X Pass

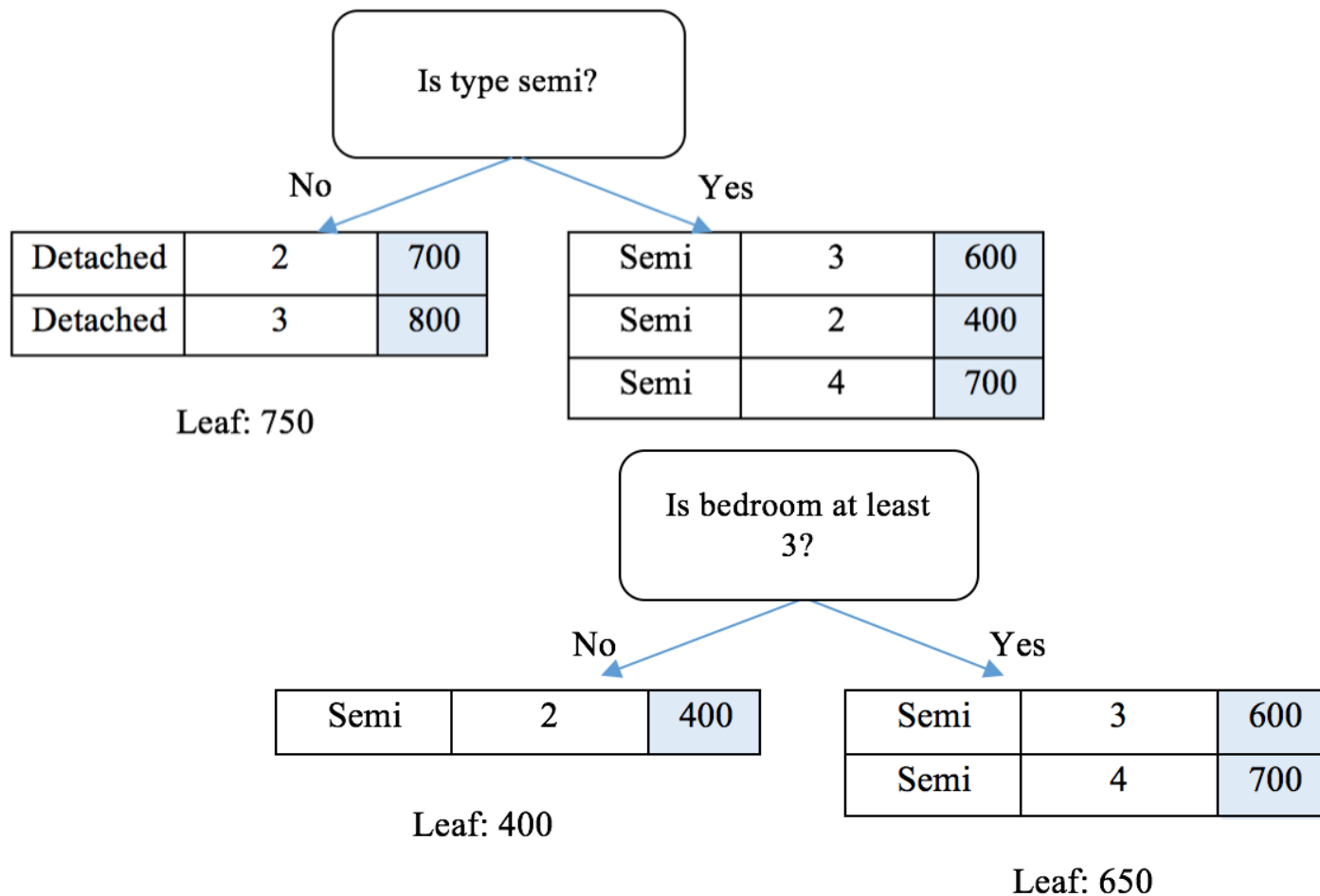
침실 수가 2개 이하 vs. 초과로 분할할 때

가중 MSE 값이 가장 작다.

|      |   |     |
|------|---|-----|
| Semi | 3 | 600 |
| Semi | 2 | 400 |
| Semi | 4 | 700 |

# 의사결정 트리 회귀 예

- 역시 마찬가지로 각 리프노드에는 **노드 데이터 평균**을 할당한다.



# 의사결정 회귀 in Python

- 사이킷런에 있는 보스턴 주택 가격 데이터에 의사결정 회귀 모델을 적용해 보자.
  - 결과값 (타겟) : 1978 보스턴 주택 가격
  - 피처 (13종)
    - 범죄율, 비소매상업지역 면적 비율, 일산화질소 농도, 주택당 방 수, 인구 중 하위 계층 비율, 인구 중 흑인 비율, 학생/교사 비율, 25,000 평방피트를 초과 거주지역 비율, 찰스강의 경계에 위치한 여부, 1940년 전 건축된 주택 비율, 방사형 고속도로까지 거리, 직업센터 거리, 재산세율

```
from sklearn import datasets

boston = datasets.load_boston()
num_test = 10 # the last 10 samples as testing set

boston.data.shape
```

(506, 13)

# 의사결정 회귀 in Python

## ■ 학습 데이터 / 테스트 데이터 분리

```
X_train = boston.data[:-num_test, :]  
y_train = boston.target[:-num_test]  
X_test = boston.data[-num_test:, :]  
y_test = boston.target[-num_test:]
```

## ■ 선형 회귀 모델 학습

```
from sklearn.tree import DecisionTreeRegressor  
  
regressor = DecisionTreeRegressor(max_depth=10, min_samples_split=3)  
regressor.fit(X_train, y_train)  
  
predictions = regressor.predict(X_test)  
print(predictions)
```

```
[18.92727273 20.9          20.9          18.92727273 20.9          28.4  
 20.73076923 24.3          26.6          20.73076923]
```

# 랜덤 포레스트 회귀 Random Forest Regression

- 분류에서 랜덤 포레스트는 모든 의사결정 트리 중 대다수가 나타내는 결과를 바탕으로 최종 결과를 생성
- 회귀에서는 모든 의사결정 트리에서 얻은 회귀 결과의 평균을 최종 결과로 만든다.
  - 각 의사 결정 트리에서 주어진 입력 데이터에 대한 리프 노드 예측 값을 계산하고, **예측값들을 평균**내서 최종 예측

# 랜덤 포레스트 회귀 in Python

```
from sklearn.ensemble import RandomForestRegressor

regressor = RandomForestRegressor(n_estimators=100,
                                  max_depth=10, min_samples_split=3)

regressor.fit(X_train, y_train)

predictions = regressor.predict(X_test)
print(predictions)
```

```
[18.96140378 20.96081003 21.55544003 19.85000734 20.96899628 25.91265703
 21.56149394 28.93643333 28.23621667 21.32265297]
```

# Support Vector Regression

---



# 서포트 벡터 회귀 Support Vector Regression

## ▪ (복습) 서포트 벡터 머신/분류기

- 입력 데이터를 서로 다른 클래스로 가장 잘 나눌 수 있는 최적의 하이퍼플레인을 찾는 알고리즘
- 하이퍼플레인은 기울기 벡터(slope)  $\mathbf{w}$ 와 절편(interception)  $b$ 로 결정
- 하이퍼플레인으로 분리된 공간 각각에서 하이퍼플레인과 가장 가까운 데이터 포인트와 하이퍼플레인 사이의 거리  $\frac{1}{\|\mathbf{w}\|}$  들이 최대가 되게 하는 하이퍼플레인을 찾는다.
  - $\|\mathbf{w}\|$ 를 최소화
  - 학습 데이터  $(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})$ 에 대해  $y^{(i)} = 1$ 이면  $\mathbf{w}\mathbf{x}^{(i)} + b \geq 1$ 이고,  $y^{(i)} = -1$ 이면  $\mathbf{w}\mathbf{x}^{(i)} + b < -1$ 를 만족한다.

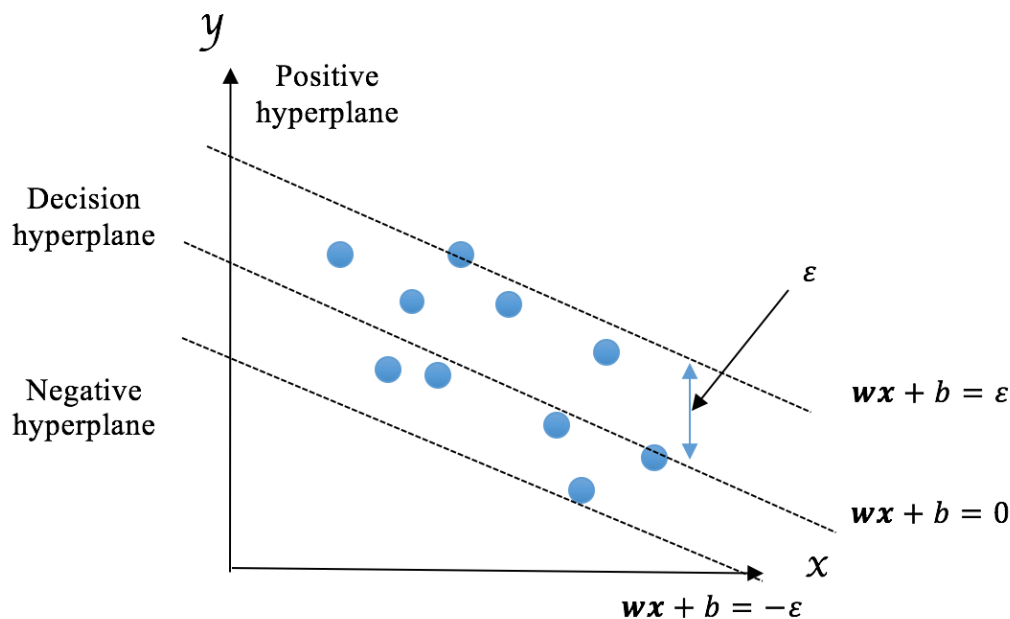
# 서포트 벡터 회귀 Support Vector Regression

## ■ 서포트 벡터 회귀

- 분류 때와 마찬가지로 회귀에서도 기울기 벡터(slope)  $\mathbf{w}$ 와 절편(interception)  $b$ 로 결정되는 하이퍼플레인을 찾는다.
- 다만, 분류 때는 데이터를 클래스 별로 잘 양분하는 하이퍼플레인이었는데, 회귀는 데이터를 잘 품을 수 있는 하이퍼플레인을 찾는다.

= 최대한 하이퍼플레인에 모든 데이터들이 가까이 있을 수 있도록 ~~

# 서포트 벡터 회귀 Support Vector Regression



- $\|\mathbf{w}\|$ 를 최소화
- 학습 데이터  $(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})$ 에 대해  $|y^{(i)} - (\mathbf{w}\mathbf{x}^{(i)} + b)| \leq \varepsilon$ 을 만족한다.

# 서포트 벡터 회귀 in Python

```
from sklearn.svm import SVR

regressor = SVR(C=0.1, epsilon=0.02, kernel='linear')

regressor.fit(X_train, y_train)

predictions = regressor.predict(X_test)

print(predictions)
```

```
[14.59908201 19.32323741 21.16739294 18.53822876 20.1960847  23.74076575
 22.65713954 26.98366295 25.75795682 22.69805145]
```

# Performance in Regression

---

# 회귀 성능 평가

- 정확히 맞췄다 못 맞췄다 기준이 명확한 분류와는 달리 회귀는 얼마나 정답과 비슷한 값으로 예측했는지 중요  
(가능한 값의 범위가 넓기 때문에 값 자체를 완전 정확하게 맞추는 것은 거의 불가능)

- 회귀 성능 평가 척도

- MSE (Mean Squared Error)  $\frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2$
- RMSE (Root Mean Squared Error)  $\sqrt{MSE}$
- MAE (Mean Absolute Error)  $\frac{1}{n} \sum_{i=1}^n |\hat{y}^{(i)} - y^{(i)}|$
- $R^2$  (R Squared) : 0~1 사이 값을 가지며 클 수록 잘 맞췄음을 의미

# 회귀 성능 평가 척도 in Python

- (당뇨병 데이터) 학습/테스트 데이터 준비

```
diabetes = datasets.load_diabetes()
num_test = 30 # the last 30 samples as testing set
X_train = diabetes.data[:-num_test, :]
y_train = diabetes.target[:-num_test]
X_test = diabetes.data[-num_test:, :]
y_test = diabetes.target[-num_test:]
```

- 학습 및 최적의 파라미터 탐색

```
param_grid = {
    "alpha": [1e-07, 1e-06, 1e-05],
    "penalty": [None, "l2"],
    "eta0": [0.001, 0.005, 0.01],
    "n_iter": [300, 1000, 3000]
}

from sklearn.model_selection import GridSearchCV
regressor = SGDRegressor(loss='squared_loss',
    learning_rate='constant')
grid_search = GridSearchCV(regressor, param_grid, cv=3)

grid_search.fit(X_train, y_train)
print(grid_search.best_params_)

{'alpha': 1e-06, 'eta0': 0.01, 'n_iter': 300, 'penalty': 'l2'}
```

# 회귀 성능 평가 척도 in Python

- 최종 테스트 결과

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

regressor_best = grid_search.best_estimator_
predictions = regressor_best.predict(X_test)
```

```
mean_squared_error(y_test, predictions)
```

2018.3155679784998

```
mean_absolute_error(y_test, predictions)
```

35.676907294371645

```
r2_score(y_test, predictions)
```

0.6082621694421424



# Lab 4-2: Linear Regression

---

# Lab

---

- NH Data Science Course

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
  - 김규태
  - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.

# Stock Prediction

---

# 유가 증권 시장과 주가

- 기업의 유가증권(주식)은 회사의 소유권을 상징
- 주식의 단일 지분은 회사 전체 자산에 대한 지분 청구 권리를 의미하며 동시에 전체 주식 수에 비례해서 기업의 이익을 배분하기도 함
- 주식 거래는 증권 거래소 같은 조직을 통해 주주와 다른 당사자들 사이에서 이뤄짐
  - 대표적 증권거래소 : 뉴욕 증권 거래소, 나스닥, 런던 증권 거래소 그룹, 상하이 증권 거래소, 홍콩 증권 거래소 등
- 주식 거래가 일어나면 수요와 공급 법칙에 따라 가격이 바뀜
  - 공급은 어느 특정 시점에 공공 투자자들이 들고 있는 주식의 수
  - 수요는 투자자들이 매수하고자 하는 주식의 수

# 주가 예측 연구

- 일반적으로 투자자는 이익을 내기 위해 주식을 낮은 가격에 매수해서 높은 가격에 매도하고 싶다.
- 하지만, 주식의 가격이 오를지 내릴지는 예측하기 어렵다.
- 주가가 변화하는데 어떤 요인과 조건들이 영향을 주는지 파악하고, 가능하면 앞날의 주가를 예측하기 위한 방법
  - 펀더멘털 분석 Fundamental Analysis
  - 기술적 분석 Technical Analysis

# 주가 예측 연구

## ■ 펀더멘털 분석 Fundamental Analysis

- 실제 주가 변동에 미치는 근본적인 영향을 이해함이 목적
- 거시적 측면: 전반적인 경제 및 산업 조건
- 미시적 측면: 회사의 재무 상태 및 관리, 경쟁사

## ■ 기술적 분석 Technical Analysis

- '왜 그렇다'는 해석은 어려워도 예측에 도움되는 패턴을 찾는 방향
- 주가의 움직임, 거래량, 과거 거래 이력 등을 분석

# 데이터: GE사의 주식 가격

- Stockmarket.csv 파일은 GE (General Electric Company )사의 주식 가격 변화를 담고 있다.
  - 2003년 1월 21일 ~ 2017년 12월 28일

```
import pandas as pd

data = pd.read_csv('stockmarket.csv')
data.head()
```

# 데이터: GE사의 주식 가격

```
import pandas as pd

data = pd.read_csv('stockmarket.csv')
data.head()
```

|   | Date       | Open      | High      | Low       | Close     | Volume   |
|---|------------|-----------|-----------|-----------|-----------|----------|
| 0 | 2003-01-21 | 23.932692 | 24.067308 | 23.067308 | 23.134615 | 22360100 |
| 1 | 2003-01-22 | 23.125000 | 23.144230 | 22.605770 | 22.644230 | 25843100 |
| 2 | 2003-01-23 | 22.740385 | 23.298077 | 22.711538 | 23.028847 | 22858800 |
| 3 | 2003-01-24 | 22.884615 | 23.028847 | 22.125000 | 22.173077 | 24768800 |
| 4 | 2003-01-27 | 22.163462 | 22.846153 | 21.855770 | 22.163462 | 29598200 |

- 시가 (Open) : 거래일 기준 최초 주가
- 종가 (Close) : 거래일 기준 마지막 주가
- 고가 (High) : 거래일 기준으로 가장 높게 거래 됐을 때 주가
- 저가 (Low) : 거래일 기준으로 가장 낮게 거래 됐을 때 주가
- 거래량 (Volume) : 주식 시장이 마감될 때까지 하루 동안 거래된 총 주식수



# 회귀를 통한 주가 예측

---

- 시가와 과거의 주식 가격 데이터를 보고  
그날의 종가를 예측하는 회귀 모델을 만들어 보자.
  - 사용 가능한 피쳐 : 시가, 과거의 "시가, 종가, 고가, 저가, 거래량"
  - 예측하고자 하는 타겟 : 종가

# 회귀를 통한 주가 예측

- 한가지 중요하게 고려해볼 사항이 있다.
  - 주가 지수에 영향을 주는 많은 변수들이 존재할 것
  - 하지만 우리가 현재 가지고 있는 피처로는 6개가 전부
    - 당일 시가, 전날 시가, 종가, 고가, 저가, 거래량

피처 수가 많다고 꼭 기계학습 성능이 좋은 것은 아니다.

하지만 문제의 복잡도에 비해 사용 가능한 피처가 현저히 적다면

당연히 기계학습 결과가 잘 나올 수 없다.

# 피쳐 엔지니어링

- 머신 러닝 알고리즘의 성능을 향상시키기 위해 기존의 피쳐를 가지고 도메인에 특화된 피쳐를 추가로 만들어내는 과정
- 충분한 **도메인 지식이 필요**하며 매우 어렵고 시간이 많이 소요되는 작업
- 실제 머신러닝 적용 상황에서 많은 경우 어떤 모델을 어떤 하이퍼 파라미터로 사용하는가 보다 피쳐 엔지니어링이 예측 성능에 큰 영향을 미친다.

# 주가 지수 예측을 위한 피처 엔지니어링

- 다양한 기간을 가지고 피처를 추출
    - 예) 하루 전날만 보는 것이 아니라 1주일, 1개월, 1년의 기간 동안의 각 피처값의 평균
  - 과거에 비해 최근 증가했는지 감소 했는지
    - 예) 과거 1년에 비해 지난주 각 피처값의 증감 비율
  - 변동성 Volatility
    - 예) 특정 기간 동안의 각 피처값의 표준 편차
  - 수익률
    - 특정 기간의 주가 지수의 손익 비율
- ... 생각해 보면  
6개 피처를 가지고  
뽑아볼 수 있는 새로운 피처들이 많다.

# 사용할 피쳐들 (1/3)

- 과거 5일 동안의 평균 증가
- 과거 1개월 동안의 평균 증가
- 과거 1년 동안의 평균 증가
- 과거 1주 동안의 평균 주가와 과거 1개월 동안의 평균 주가 사이의 비율
- 과거 1주 동안의 평균 주가와 과거 1년 동안의 평균 주가 사이의 비율
- 과거 1개월 동안의 평균 주가와 과거 1년 동안의 평균 주가 사이의 비율
- 과거 5일 동안의 평균 거래량
- 과거 1개월 동안의 평균 거래량
- 과거 1년 동안의 평균 거래량
- 과거 1주 동안의 평균 거래량과 과거 1개월 동안의 평균 거래량 사이의 비율
- 과거 1주 동안의 평균 거래량과 과거 1년 동안의 평균 거래량 사이의 비율
- 과거 1개월 동안의 평균 거래량과 과거 1년 동안의 평균 거래량 사이의 비율

# 사용할 피쳐들 (2/3)

- 과거 5일 동안 평균 증가의 표준 편차
- 과거 1개월 동안 평균 증가의 표준 편차
- 과거 1년 동안 평균 증가의 표준 편차
- 과거 1주 동안 주가의 표준 편차와 과거 1개월 동안 주가의 표준 편차 사이의 비율
- 과거 1주 동안 주가의 표준 편차와 과거 1년 동안 주가의 표준 편차 사이의 비율
- 과거 1개월 동안 주가의 표준 편차와 과거 1년 동안 주가의 표준 편차 사이의 비율
- 과거 5일 동안 평균 거래량의 표준 편차
- 과거 1개월 동안 평균 거래량의 표준 편차
- 과거 1년 동안 평균 거래량의 표준 편차
- 과거 1주 동안 거래량의 표준 편차와 과거 1개월 동안 거래량의 표준 편차 사이의 비율
- 과거 1주 동안 거래량의 표준 편차와 과거 1년 동안 거래량의 표준 편차 사이의 비율
- 과거 1개월 동안 거래량의 표준 편차와 과거 1년 동안 거래량의 표준 편차 사이의 비율

# 사용할 피쳐들 (3/3)

- 전일 일일 수익률
- 1주전 주간 수익률
- 1개월 전 월간 수익률
- 과거 1년 기준 연간 수익률
- 과거 1주 동안 일일 수익률의 이동 평균  
(각 거래일의 다음 일주일 값들의 평균의 평균)
- 과거 1개월 동안 일일 수익률의 이동 평균  
(각 거래일의 다음 1개월 값들의 평균의 평균)
- 과거 1년 동안 일일 수익률의 이동 평균  
(각 거래일의 다음 1년 값들의 평균의 평균)
- + 기존 피쳐 : 시가, 전일 시가, 전일 종가, 전일 고가, 전일 저가, 전일 거래량

# 피처 엔지니어링

## ■ 피처 엔지니어링 함수 구현

```
def generate_features(df):  
  
    df_new = pd.DataFrame()  
  
    df_new['open'] = df['Open']  
    df_new['open_1'] = df['Open'].shift(1)  
    df_new['close_1'] = df['Close'].shift(1)  
    df_new['high_1'] = df['High'].shift(1)  
    df_new['low_1'] = df['Low'].shift(1)  
    df_new['volume_1'] = df['Volume'].shift(1)
```

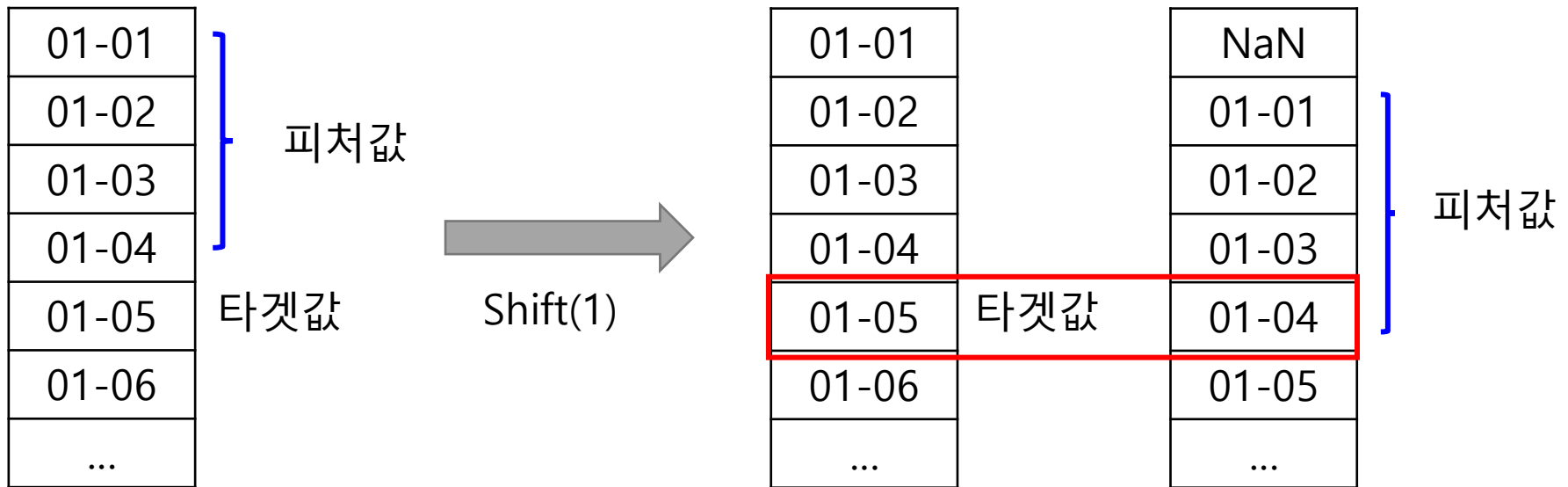
Dataframe.shift( k )는

Dataframe의 제일 앞에 행을 k개 만큼 밀어서 (NaN으로 채움)  
새로운 Dataframe을 만드는 함수다.



# 피쳐 엔지니어링

## ■ 피쳐 엔지니어링 함수 구현



1월 5일에 주가를 예측하기 위해 그 전날인 1월 4일까지의 데이터를 봐야한다.

하지만 타겟값과 피쳐값이 다른 줄에 걸쳐 있기 때문에

이를 맞추기 위해 shift 함수를 사용한다.

# 피처 엔지니어링

## ■ 피처 엔지니어링 함수 구현

```
df_new['avg_price_5'] = df['Close'].rolling(window=5).mean().shift(1)
df_new['avg_price_30'] = df['Close'].rolling(window=21).mean().shift(1)
df_new['avg_price_365'] = df['Close'].rolling(window=252).mean().shift(1)
```

Dataframe.rolling 함수를 사용하면  
7일전 종가 평균, 30일전 종가 평균, 1년 전 종가 평균을  
쉽게 구할 수 있다.

# 피쳐 엔지니어링

## ■ 피쳐 엔지니어링 함수 구현

*Pandas Dataframe의 Rolling 함수 작동 예)*

|       |     |
|-------|-----|
| 01-01 | 1   |
| 01-02 | 2   |
| 01-03 | 1   |
| 01-04 | 4   |
| 01-05 | 3   |
| 01-06 | 2   |
| ...   | ... |



Rolling(5).mean()

|       |     |
|-------|-----|
| 01-01 | NaN |
| 01-02 | NaN |
| 01-03 | NaN |
| 01-04 | NaN |
| 01-05 | 2.2 |
| 01-06 | 2.4 |
| ...   | ... |



Shift(1)

|     |
|-----|
| NaN |
| NaN |
| NaN |
| NaN |
| NaN |
| 2.2 |
| ... |

# 피처 엔지니어링

- 피처 엔지니어링 함수 구현

```
f_data = generate_features(data)
```

코드가 긴 관계로

자세한 내용은 코드 파일 참고

```
def generate_features(dataframe):
```

# 학습, 테스트 데이터 생성

- 2016년까지 데이터를 학습에 사용하고 2017년 데이터를 테스트에 사용

```
train_data = f_data[ f_data['date'] <= '2016-12-31']  
test_data = f_data[ f_data['date'] > '2017-01-01']
```

- 학습, 테스트 데이터에 대해서 피처와 타겟값을 분리

```
x_train_data = train_data.drop(['close', 'date'], axis=1)  
y_train_data = train_data['close']  
x_test_data = test_data.drop(['close', 'date'], axis=1)  
y_test_data = test_data['close']
```

# 정규화 Normalization

- 여러 피처가 존재하는데 각 피처의 도메인마다 주로 발생 가능한 값의 범위가 다를 수 있다.
- 만약 특정 피처가 굉장히 큰 값을 가지고 있고, 다른 피처는 아주 작은 값을 가진다면 같은 조건에서 학습을 했을 때 **작은 값을 가지는 피처들이 상대적으로 무시**될 수 있다.
- 이러한 문제를 극복하기 위한 방법이 **정규화**
  - Min-Max 정규화 :  $(X - \text{Min}(X)) / (\text{Max}(X) - \text{Min}(X))$
  - 표준정규화 :  $(X - \text{Mean}(X)) / \text{Std}(X)$

# 정규화 Normalization

- 표준 정규화 적용

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()  
X_scaled_train = scaler.fit_transform(X_train_data)  
X_scaled_test = scaler.transform(X_test_data)
```

# 학습 및 파라미터 탐색

## ■ 선형 회귀 적용

```
from sklearn.linear_model import SGDRegressor
from sklearn.model_selection import GridSearchCV

param_grid = {
    "alpha": [3e-06, 1e-5, 3e-5],
    "eta0": [0.01, 0.03, 0.1],
}

lr = SGDRegressor(penalty='l2', n_iter=1000)

grid_search = GridSearchCV(lr, param_grid, cv=5, scoring='neg_mean_absolute_error')

grid_search.fit(X_scaled_train, y_train_data)

print(grid_search.best_params_)

{'alpha': 3e-06, 'eta0': 0.01}
```



# 테스트 데이터 예측 및 평가

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

lr_best = grid_search.best_estimator_

predictions = lr_best.predict(X_scaled_test)

print('MSE: {0:.3f}'.format(mean_squared_error(y_test_data, predictions)))

print('MAE: {0:.3f}'.format(mean_absolute_error(y_test_data, predictions)))

print('R^2: {0:.3f}'.format(r2_score(y_test_data, predictions)))
```

```
MSE: 0.096
MAE: 0.260
R^2: 0.927
```

# 다른 알고리즘들도 적용

- Random Forest Regression 적용

```
param_grid = {  
    "max_depth": [30, 50],  
    "min_samples_split": [3, 5, 10],  
}  
rf = RandomForestRegressor(n_estimators=1000)  
grid_search = GridSearchCV(rf, param_grid, cv=5, scoring='neg_mean_absolute_error')  
grid_search.fit(X_scaled_train, y_train_data)
```

- Support Vector Regression 적용

```
param_grid = {  
    "C": [1000, 3000, 10000],  
    "epsilon": [0.00001, 0.00003, 0.0001],  
}  
svr = SVR(kernel='linear')  
grid_search = GridSearchCV(svr, param_grid, cv=5, scoring='neg_mean_absolute_error')  
grid_search.fit(X_scaled_train, y_train_data)
```

# Lab 4-3: Competition

---

# Lab

---

- NH Data Science Course

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
  - 김규태
  - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.